

Git e GitHub - Trabalho em Equipe com Branches

Cenário

Projeto hospedado no GitHub: `eutealugo`

Equipe:

- **Aluno A** → CSS / Estilização (Branch: `estilo`)
 - **Aluno B** → JavaScript (Branch: `javascript`)
-

1. Clonar o Projeto

bash

```
cd C:\Users\gil_m\Downloads

git clone https://github.com/profgilmarfreire/eutealugo.git
eutealugo-estilo

cd eutealugo-estilo
```

Use o código com cuidado.

2. Abrir no VS Code

bash

```
code .
```

Use o código com cuidado.

3. Verificar Configuração do Git

bash

```
git config --global user.name

git config --global user.email
```

Use o código com cuidado.

Se não estiver configurado:

bash

```
git config --global user.name "Nome do Aluno"
git config --global user.email "aluno@email.com"
```

Use o código com cuidado.

4. Atualizar Projeto (Antes de Iniciar)

bash

```
git pull origin main
```

Use o código com cuidado.

5. Criar e Entrar na Branch

- **Aluno CSS:** `git switch -c estilo`
- **Aluno JavaScript:** `git switch -c javascript`

6. Confirmar Branch Atual

bash

```
git branch
```

Use o código com cuidado.

(O asterisco *** indica a branch ativa). [\[1\]](#)

7. Realizar Alterações

Edite os arquivos necessários no VS Code.

8. Consultar Alterações

bash

```
git status
```

Use o código com cuidado.

9. Adicionar Arquivos ao Staging

bash

```
git add .
```

Use o código com cuidado.

10. Cancelar o git add (Se necessário)

bash

```
git rm --cached -r .
```

Use o código com cuidado.

11. Criar Commit

bash

```
git commit -m "Cria estilização da página inicial"
```

Use o código com cuidado.

12. Ver Histórico de Commits

bash

```
git log --oneline
```

Use o código com cuidado.

13. Publicar a Branch no GitHub (Primeira Vez)

- **Aluno CSS:** `git push -u origin estilo`
- **Aluno JavaScript:** `git push -u origin javascript`

14. Próximos Envios

bash

```
git push
```

Use o código com cuidado.

15. Unificar o Trabalho (O Git Merge)

Após os alunos enviarem suas branches para o GitHub, o código precisa ser juntado na branch `main`. Isso pode ser feito de duas formas: [\[1\]](#)

Abordagem A: Mesclagem Local (Via Terminal)

Geralmente feita pelo líder do projeto ou pelo próprio aluno após finalizar a tarefa.

Volte para a branch principal:

bash

```
git switch main
```

1. Use o código com cuidado.

Atualize a sua main local com o GitHub (garanta que tem o código mais recente):

bash

```
git pull origin main
```

2. Use o código com cuidado.

Traga as alterações da branch do aluno para a main:

bash

```
git merge estilo
```

3. Use o código com cuidado.
(Substitua `estilo` por `javascript` quando for juntar o outro trabalho).

Envie a main atualizada de volta para o GitHub:

bash

```
git push origin main
```

4. Use o código com cuidado.

[\[1, 2, 3, 4\]](#)

Abordagem B: Via Pull Request (Recomendado para Equipes)

Em vez de fazer o merge direto no terminal, os alunos pedem autorização no GitHub:

1. O aluno entra no repositório no GitHub.
2. Clica no botão **"Compare & pull request"** que aparece na tela.
3. Escreve o que foi feito e clica em **"Create pull request"**.
4. O professor ou colega revisa o código e clica em **"Merge pull request"**.

Depois que o GitHub aprovar, todos os alunos devem rodar na máquina deles:

bash

```
git switch main
```

```
git pull origin main
```

5. Use o código com cuidado.

[\[1, 2, 3, 4, 5\]](#)

16. Resolvendo Conflitos (Se acontecerem)

Se o Aluno A e o Aluno B mexerem na **mesma linha** do mesmo arquivo, o Git vai travar o merge e avisar: *CONFLICT (content): Merge conflict in...*

Como resolver:

1. Abra o arquivo conflitado no VS Code.
2. O VS Code mostrará o código dos dois alunos destacados em verde e azul.
3. Escolha uma das opções na tela:
 - *Accept Current Change* (Manter o que já estava na main).
 - *Accept Incoming Change* (Aceitar o que está vindo da branch).
 - *Accept Both Changes* (Manter o código dos dois). [\[1\]](#)
4. Salve o arquivo. [\[1\]](#)

Conclua o merge pelo terminal:

bash

```
git add .
```

```
git commit -m "Resolve conflito de mesclagem"
```

```
git push origin main
```

5. Use o código com cuidado.

[\[1\]](#)

FLUXO RESUMIDO (Com Merge Local)

bash

```
git clone URL
```

```
git switch -c estilo
```

```
# ... faz alterações ...
```

```
git add .
```

```
git commit -m "Ajustes de estilo"
```

```
git push -u origin estilo
```

```
# Hora de juntar na main:
```

```
git switch main
```

```
git pull origin main
```

```
git merge estilo
```

```
git push origin main
```

Use o código com cuidado.